

# **Community Detection in Networks**

**An Analysis of the Louvain Algorithm and the Fast Louvain Mechanism**

## **IE203 : Data Structures & Algorithms**

Saanvi TS - 24B4501

Shivam Soni - 24B4525

Anant Asati - 24B4530

# Contents

|          |                                                              |          |
|----------|--------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction to Community Detection</b>                   | <b>2</b> |
| 1.1      | What is a Community? . . . . .                               | 2        |
| 1.2      | Why Community Detection? . . . . .                           | 2        |
| <b>2</b> | <b>The Modularity Metric (<math>Q</math>) for Evaluation</b> | <b>2</b> |
| <b>3</b> | <b>The Standard Louvain Algorithm (LVA)</b>                  | <b>3</b> |
| 3.1      | The Two-Phase Iterative Process . . . . .                    | 3        |
| 3.2      | Time Complexity . . . . .                                    | 4        |
| <b>4</b> | <b>Inefficiencies and Structural Limitations of LVA</b>      | <b>5</b> |
| 4.1      | Computational Bottlenecks . . . . .                          | 5        |
| 4.2      | Structural Limitations: The Failure of Modularity . . . . .  | 6        |
| <b>5</b> | <b>The Fast Louvain Mechanism (FLA)</b>                      | <b>6</b> |
| 5.1      | The Improvement Strategy . . . . .                           | 6        |
| 5.1.1    | Improvement 1: Dynamic Iteration Optimization . . . . .      | 6        |
| 5.1.2    | Improvement 2: Early Stopping Criterion . . . . .            | 6        |
| 5.1.3    | Improvement 3: Splitting Local Tree Structures . . . . .     | 7        |
| <b>6</b> | <b>Simulation and Benchmark Results</b>                      | <b>8</b> |
| 6.1      | Efficiency Gain . . . . .                                    | 8        |
| <b>7</b> | <b>Performance Across Datasets and Quality Impact</b>        | <b>8</b> |
| <b>8</b> | <b>Conclusions on Fast Louvain Performance</b>               | <b>9</b> |

Project Repository

All simulation code, demonstrations, readme and algorithms are available on GitHub:  
<https://github.com/Soni-Shivam/Community-Detection>

# 1 Introduction to Community Detection

## 1.1 What is a Community?

A community is formally defined as a subset of nodes within a network where the **internal connections are stronger or more frequent** than the external connections to other nodes in the network.

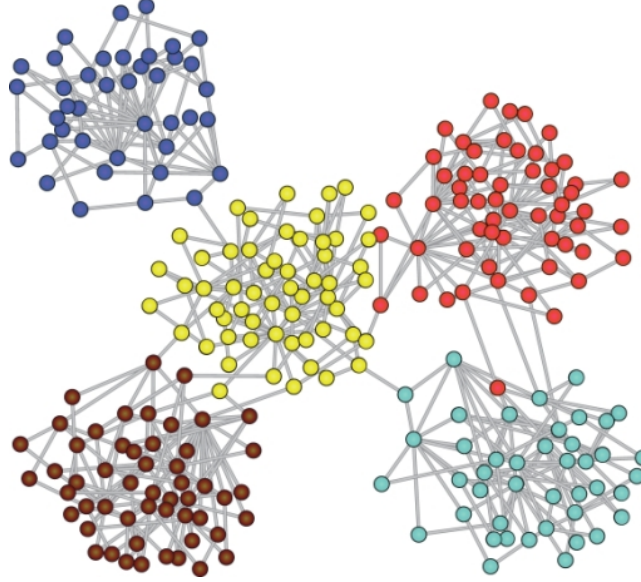


Figure 1: Visual representation of a network with distinct community structures.

## 1.2 Why Community Detection?

Community detection is critical for uncovering hidden functional or structural patterns within complex systems. And it has applications in diverse domains, including Information Systems (for recommendation engines), Biological Sciences (for analyzing gene and protein interaction networks), and Computer Science (for fraud detection and network protocol design).

# 2 The Modularity Metric ( $Q$ ) for Evaluation

The strength of a community partition is quantified using **Modularity** ( $Q$ ), which represents the strength of division into communities. It allows us to assess a 'good' community distribution.

The formal equation for Modularity for a partition  $\mathcal{C}$  is:

$$Q(\mathcal{C}) := \frac{1}{2m} \sum_{c \in \mathcal{C}} \sum_{u \in c} \sum_{v \in c} \left( A_{uv} - \frac{k_u k_v}{2m} \right) \quad (1)$$

Where:

- $2m$  is the total number of edges in the network (sum of degrees).
- $A_{uv}$  is the weight of the edge between nodes  $u$  and  $v$
- $k_u$  and  $k_v$  are the degrees of nodes  $u$  and  $v$ , respectively.
- The term  $\frac{k_u k_v}{2m}$  represents the **expected number of edges** between  $u$  and  $v$  in the random baseline null model.

### 3 The Standard Louvain Algorithm (LVA)

The Louvain Algorithm is a **greedy, hierarchical, agglomerative** algorithm specifically designed for **Modularity Maximization** ( $Q$ ). It addresses the question of how to detect communities by using a greedy approach.

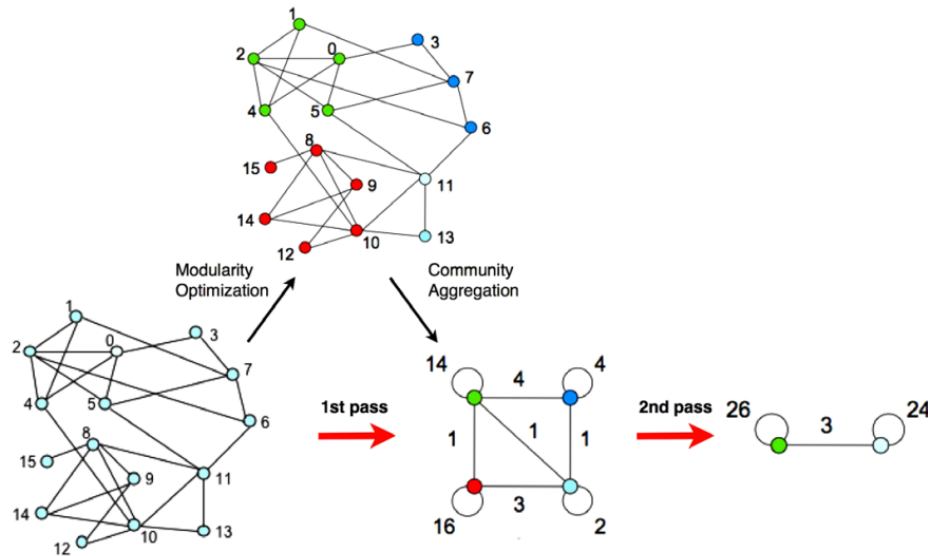


Figure 2: The structure of the Louvain Algorithm: Nodes merge to maximize local modularity, then aggregate into super-nodes.

#### 3.1 The Two-Phase Iterative Process

The algorithm works by iteratively optimizing the network partition.

##### Phase 1: Local Modularity Optimization (Greedy)

| Step           | Description                                                                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Initialization | Every single node is initially treated as its own community. For a graph with 9 nodes (0 to 8), the initial community set $C$ is $\{\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}\}$ . |
| Calculation    | For each node $i$ , the algorithm calculates the <b>modularity gain</b> ( $\Delta Q$ ) that would result from moving it to a neighboring community.                                                    |
| Movement       | Node $i$ is moved to the neighboring community that provides the <b>largest positive gain</b> in $Q$ .                                                                                                 |
| Termination    | This phase is repeated until <b>no single node move</b> can further increase the overall modularity score.                                                                                             |

The governing equation for the Modularity Gain ( $\Delta Q$ ) when moving a node  $i$  into a community  $C$  is:

$$\Delta Q = \left[ \frac{\sum_{in} + k_{i,in}}{2m} - \left( \frac{\sum_{tot} + k_i}{2m} \right)^2 \right] - \left[ \frac{\sum_{in}}{2m} - \left( \frac{\sum_{tot}}{2m} \right)^2 \right] \quad (2)$$

Where:

- $\sum_{in}$  is the sum of the weights of the edges inside community  $C$  before moving  $i$ .
- $k_{i,in}$  is the sum of the weights of the edges between node  $i$  and nodes in community  $C$ .
- $\sum_{tot}$  is the sum of the weights of the edges attached to nodes in community  $C$  before moving  $i$ .
- $k_i$  is the sum of the weights of the edges attached to node  $i$ .
- $2m$  is the sum of the weights of all edges in the network.

## Phase 2: Graph Aggregation (Network Coarse Graining)

The second phase leverages the self-similar nature of complex networks by constructing a new network where communities found in Phase 1 become nodes.

| Mechanism           | Description                                                                                                                                                                                                                       |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Super-Node Creation | Nodes belonging to the same community are <b>aggregated</b> into a single "super-node" representing that community.                                                                                                               |
| Edge Reweighting    | The weight of a edge between two super-nodes is the <b>sum of the weights</b> of the edges between their constituent nodes. Links within the original community become <b>self-loops</b> for the super-node.                      |
| Iterative Hierarchy | The algorithm <b>reapplies Phase 1</b> to this new weighted network. This cycle (Phase 1 + Phase 2) constitutes a "pass" and repeats until maximum modularity is attained, naturally building a <b>hierarchy of communities</b> . |

## 3.2 Time Complexity

The major advantage of the Louvain algorithm is its **remarkable speed and scalability** for large networks.

- **Complexity:** Typically cited as **near-linear**,  $O(M \cdot \log N)$
- $M$  is the number of edges.
- $N$  is the number of nodes.

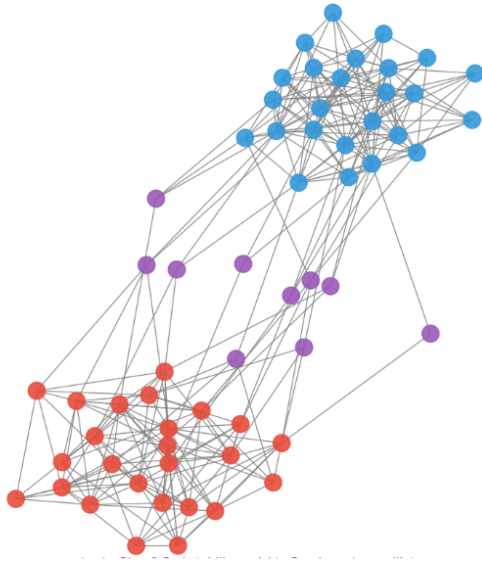
## 4 Inefficiencies and Structural Limitations of LVA

### 4.1 Computational Bottlenecks

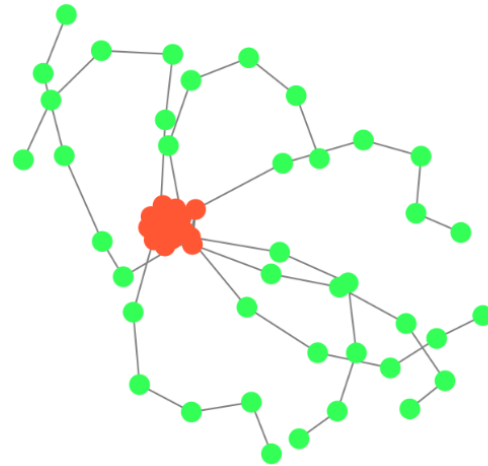
The Louvain Algorithm (LVA)’s iterative, greedy nature results in significant wasted computation, particularly for networks with millions of edges where the constant factor and hidden costs become major bottlenecks.

Table 1: LVA Bottlenecks

| Bottleneck                            | Issue Description                                                                                                                                                                                                           | Computational Waste                                                |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Redundant Iteration Passes            | The algorithm re-traverses all nodes in every iteration of Phase 1. Calculation is <b>repeated needlessly</b> once a pair of communities reaches a “local stable structure” (i.e., no $\Delta Q$ is possible between them). | Repeated, unnecessary calculations of $\Delta Q$ for stable pairs. |
| Slow Convergence on Sparse Structures | Many real-world networks contain sparse leaf nodes and long chains (local tree structures). Including these structurally unambiguous nodes in the main iteration is inefficient.                                            | Wastes time and slows the convergence of the entire graph.         |
| Wasted Final Moves                    | The stopping criterion requires <b>zero node moves</b> to halt the iteration. The final passes yield <b>negligible modularity change</b> .                                                                                  | Requires a full pass over the entire graph for marginal gain.      |



(a) Redundant Iteration Passes:



(b) Sparse Structures:

## 4.2 Structural Limitations: The Failure of Modularity

Even with perfect efficiency, the LVA is fundamentally limited by the Modularity  $Q$  metric it optimizes. This is a critical flaw of the method.

Table 2: Structural Limitations of Modularity

| Structural Limitation    | Description                                                                                                                                                                                                                                       | Resulting Issue                                                                                                          |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| The Resolution Limit     | Modularity is calculated against a <b>global null model</b> (a random graph). This global comparison prevents the algorithm from detecting small, tightly-knit communities when they are only <b>weakly connected</b> to the rest of the network. | Small, genuine sub-communities are <b>merged into larger clusters</b> , ultimately hiding the network's finer structure. |
| Disconnected Communities | Phase 1 is a purely greedy optimization. A move that increases $Q$ (globally beneficial) might locally <b>break a community's internal connectivity</b> .                                                                                         | A node acting as a "bridge" in its community may move, leaving the original community internally disconnected.           |

## 5 The Fast Louvain Mechanism (FLA)

### 5.1 The Improvement Strategy

The Fast Louvain Mechanism (FLA) focuses on three main improvements to overcome the LVA's computational bottlenecks.

#### 5.1.1 Improvement 1: Dynamic Iteration Optimization

- **Goal:** Eliminate  $\Delta Q$  recalculations for stable communities.
- **Mechanism:** The algorithm tracks which community pairs were unchanged in an iteration.
- Future passes skip considering node movements between these identified "**local stable structures**".

#### 5.1.2 Improvement 2: Early Stopping Criterion

- **Goal:** Avoid spending time on the marginal gains  $\Delta Q$  in the final passes.
- **Mechanism:** Halt the Phase 1 iteration if the **node migration rate** drops below a negligible threshold (e.g., 0.1% of total nodes), rather than waiting for absolute stability.

### 5.1.3 Improvement 3: Splitting Local Tree Structures

- **Goal:** Remove redundant nodes from the costly iterative process to improve efficiency and the partitioning effect.
- **Standard LVA Issue:** The standard algorithm wastes cycles calculating modularity for every node in long chains repeatedly, even though they will always belong to the community of the node they are attached to.
- **Mechanism:**
  1. Identify all **tree-like components** (leaf nodes, chains). **Remove** them (pruning).
  2. Run the Louvain process on the smaller, **dense core**.
  3. **Reattach** the trees to the resulting communities.
- **Mathematical Justification:** Removing these sparse structures is mathematically safe because the modularity change caused by splitting or merging them is **negligible**
- **Implementation:** Achieved using a **queue-based cascading removal strategy** with a complexity of  $O(V + E)$  and reconstructing the complete partition using a stack with  $O(m)$  complexity (where  $m$  is the number of nodes pruned).

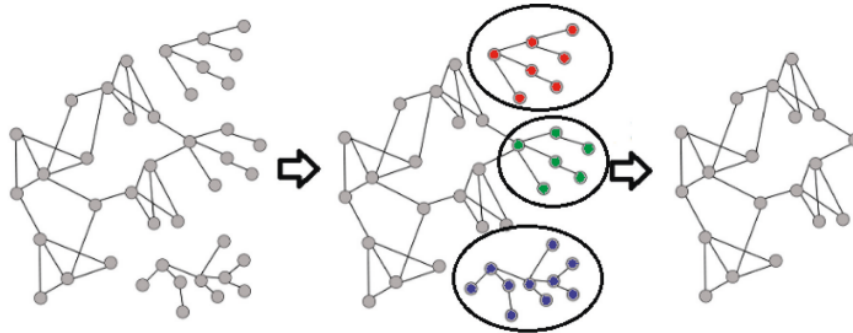


Figure 4: of Tree Splitting: Sparse leaf nodes are pruned, processed separately, and reattached, reducing the load on the core iterative process.

These improvements drastically reduce the computational load per pass.

Table 3: Comparison of Complexity per Pass

| Metric              | Standard Louvain                                      | Fast Louvain (Improved)                                                         |
|---------------------|-------------------------------------------------------|---------------------------------------------------------------------------------|
| Complexity Per Pass | $O(k \cdot N)$                                        | $O(k \cdot N_{active})$                                                         |
| Variables           | $k$ = average degree,<br>$N$ = total number of nodes. | $N_{active}$ = number of nodes in <b>topologically stationary communities</b> . |

## 6 Simulation and Benchmark Results

Our team simulated both LVA and FLA using C++ utilizing multiple real-world network datasets to evaluate performance and quality.

- **Dataset:** Facebook user connections
- **Network Size:** 4,039 Nodes and over 17,000 connections

The benchmark clearly demonstrates the performance gap between the two algorithms.

Table 4: Benchmark Results: LVA vs. FLA

| Metric                         | Standard Louvain                 | Fast Louvain                    |
|--------------------------------|----------------------------------|---------------------------------|
| Total Steps (Calc $\Delta Q$ ) | 232,687                          | 150,255                         |
| Final Modularity ( $Q$ )       | 0.86189                          | 0.85817                         |
| Initial Phase 2 Aggregation    | 4,039 nodes into 103 communities | 4,039 nodes into 99 communities |
| Final Phase 2 Aggregation      | 103 nodes into 72 communities    | 99 nodes into 63 communities    |

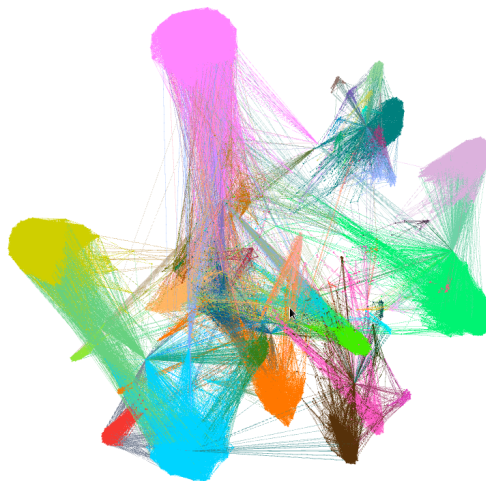


Figure 5: Community detection results as visualised on Gephi. Beautiful isn't it!?

### 6.1 Efficiency Gain

The application of the Fast Louvain Mechanism yielded a substantial computational improvement: **35.43% fewer calculations** & **Saved 82,432 steps** with negligible change in modularity.

## 7 Performance Across Datasets and Quality Impact

Experimenting on multiple datasets available on Stanford Large Network Dataset Collection we get the following results. Our largest run was using the Twitter social networks dataset having 81,306 nodes and 170k+ edges which interestingly had an higher modularity using the FLA!

Table 5: Performance across Datasets

| Dataset                | Orig.<br>Time<br>(ms) | Fast<br>Time<br>(ms) | Old<br>Mod | New<br>Mod | Decr.<br>in<br>Steps | Eff.<br>Gain |
|------------------------|-----------------------|----------------------|------------|------------|----------------------|--------------|
| Twitter Social Circles | 109,624               | 94,830               | 0.66774    | 0.68651    | 766,975              | 13.78%       |
| Facebook (small)       | 1,196                 | 765                  | 0.86540    | 0.86889    | 44,821               | 22.34%       |

## 8 Conclusions on Fast Louvain Performance

The Fast Louvain Algorithm (FLA) is a clear advancement over the traditional Louvain Algorithm (LVA).

Table 6: Conclusions

| Aspect             | Conclusion for Fast Louvain Algorithm (FLA)                                                                                                                                                                                               |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Efficiency Gains   | Delivers a dramatic improvement in computational efficiency, achieving runtime reductions of <b>15%–30%</b> on large-scale, sparse networks. This is driven by optimizing the iterative process and pre-processing local tree structures. |
| Quality Assessment | The FLA successfully maintains or slightly improves the final Modularity $Q$ score. Its tree splitting strategy results in a <b>more uniform and refined partition</b> with higher community aggregation overall.                         |
| Scalability        | Its ability to handle large-scale datasets while reducing computation time by nearly 20% establishes it as a <b>superior and more scalable alternative</b> to the classical LVA for analyzing modern complex networks.                    |

## References

[1] V. D. Blondel, J. L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics: Theory and Experiment*, 2008. Available: [arXiv:0803.0476](#)

[2] “Comparative Analysis of Community Detection Algorithms on the SNAP Social Circles Dataset,” *arXiv preprint*, 2025. Available: [arXiv:2502.04341v1](#)

[3] J. Leskovec and A. Krevl, “SNAP Datasets: Stanford Large Network Dataset Collection,” Stanford Network Analysis Project. Available: [snap.stanford.edu/data](#)

[4] “Discovering Communities,” Video Lecture Available: [YouTube Link](#)